PSEL4 keeps ZX rewriting only where it beats pytket level 3 on Quantinuum H2

PSEL4 is a drop-in pytket pass selector. By construction it never costs more HQC than pytket level 2 or level 3 on a circuit where both pass its checks. On the 17 arithmetic and oracle circuits whose output is proven it saves 0.80 % against level 3.

Team Learned ZX Rewriting

C.-F. Kam, A. Mahasinghe, D. De Silva, N. Karunarathne, F. Cadet, M. Bessafi, K. De Silva

Quantinuum Singapore Grand Challenge, Submission 1
October 2026

ZX rewriting before pytket can raise the cost of an H2 run

A run on Quantinuum H2 is billed in HQC.

$$\mathrm{HQC} = 5 + C\left(N_{1q} + 10\,N_{2q} + 5\,N_{\mathrm{SPAM}}\right)/5000$$

C is the shot count. N_{1q} counts PhasedX gates. N_{2q} counts ZZPhase and ZZMax gates. N_{SPAM} counts initialisations, measurements and resets. Rz gates are free.

- ZX rewriting (PyZX teleport_reduce) placed before pytket's Quantinuum pass can lower this cost or raise it.
- Run blindly before pytket level 2, it costs 6.27 % more HQC on 180 Trotter circuits and 22.1 % more on 3 QFT adders.
- Our baseline is pytket level 3. On the 20 family C circuits level 3 alone costs 23.16 % less than the default level 2.
- On draper_adder_16 teleport_reduce before level 2 costs 1,385.0 HQC against 906.8 for level 2.
- pytket's own ZX pass `ZXGraphlikeOptimisation` is documented as one that "may increase the cost of the circuit".
- A team with a fixed HQC budget cannot tell in advance which circuits gain. **[O]**

PSEL4 answers one question per circuit for a team on a fixed HQC budget

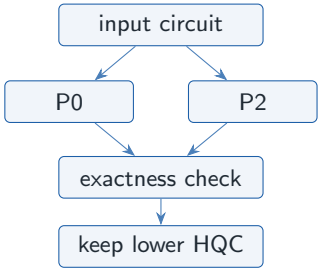
- **User.** A team that runs arithmetic and oracle subroutines on Quantinuum H2 from a fixed HQC budget. **[O]**
- **Decision.** Should this circuit go through ZX rewriting before pytket's Quantinuum pass?
- **Answer.** PSEL4 compiles the circuit at level 2 and level 3, each with and without teleport_reduce. It keeps the cheapest result that passes its checks, which run up to 20 qubits only. Both levels are always candidates.
- **Guarantee.** PSEL4 never costs more HQC than level 2 or level 3 when both pass its checks. This is a design property, not a measured result.
- **Why per circuit.** On the 60 XXZ circuits P2 alone raises the total HQC by 12.33 %. PSEL keeps P0 on all 60.
- **Finals roadmap.** Learned ZX rewriting is the finals goal. A model will predict which ZX rewrites lower HQC. The P0 and P2 rows of the PSEL ledger are its training data. **[O]**

No user interview or survey is part of this submission.

Untagged statements are [F] facts. [O] marks a judgement.

PSEL compiles each circuit twice and keeps the cheaper result

- **P0.** pytket-quantinuum 0.59.3 default pass at optimisation level 2. The baseline.
- **P2.** PyZX teleport_reduce, then P0.
- **PSEL** drops a candidate that fails its exactness check and keeps the one with the lower HQC. Ties go to the lower two-qubit count, then to P0.
- **Official baseline.** Our P0 and P2 give the same counts as the official pytket-quantinuum pass on all 445 compiled rows of families A, B1, B1x and B2. Family C was not compiled with the official pass.



`pipelines/pass_select.py`

Assessment [O]. PSEL is pytket's PassSelector pattern with HQC as the metric. The contribution is the measured cost ledger and the guard result, not a new selection mechanism.

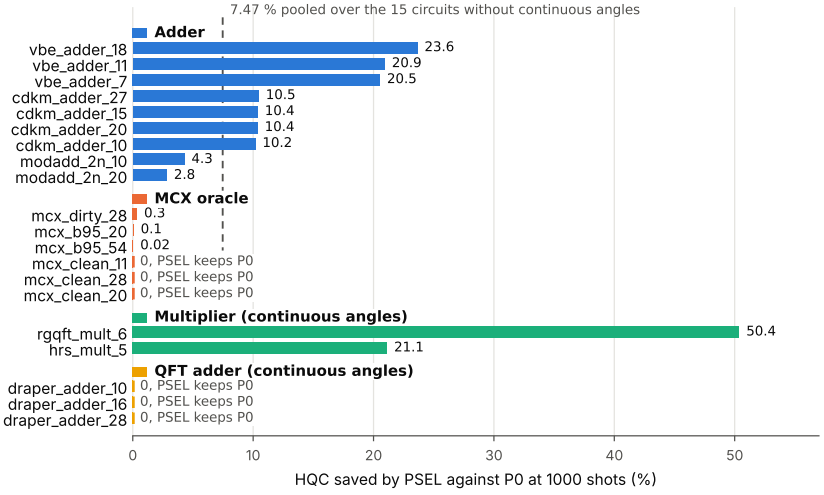
PSEL saves 7.47 % on 15 family C circuits without continuous angles

Family C	Circuits	Qubits	Picks P2	HQC P0	HQC PSEL	Saved	Saved %
Adder	9	20–56	9	5777.0	5031.4	745.6	12.9
Grover oracle (MCX)	6	21–56	3	4242.6	4239.8	2.8	0.1
No continuous angles	**15**	**20–56**	**12**	**10019.6**	**9271.2**	**748.4**	**7.47**
Multiplier, continuous angles	2	21–24	2	9737.0	6381.0	3356.0	34.5
QFT adder, continuous angles	3	20–56	0	4778.4	4778.4	0.0	0.0
All of family C	20	20–56	14	24535.0	20430.6	4104.4	16.7

HQC at 1000 shots. Family C is count-only apart from one H2-2E run.

- A PyZX miter reduction proves the PSEL output equal to its input on 18 of the 20 circuits. All 15 without continuous angles are proven as a full unitary up to global phase.
- hrs_mult_5 is inconclusive. The miter of the P0 pick of draper_adder_28 timed out. PSEL itself runs no exactness check above 20 qubits.
- The two multipliers carry 81.8 % of the 4104.4 HQC saved on family C.

Adders and multipliers carry the savings on family C



HQC at 1000 shots. A zero bar means PSEL keeps P0. Data `value/family_C_selector.csv`.

The selector avoids the losses of always running ZX

Guard against always running P2

- 180 Trotter circuits. P2 alone would cost 1,030.4 HQC or 6.27 % more than P0.
- PSEL avoids 1,076.4 HQC there against always running P2. It still saves 46.0 HQC against P0.
- 3 QFT adders. P2 is dearer on all three. PSEL keeps P0 and avoids 1,057.2 HQC.
- PSEL never raises the two-qubit count against P0 on the 223 circuits of families A, B1, B1x and B2.

Clifford+T benchmarks B1

- 15 circuits with $n \leq 12$. PSEL picks P2 on 14.
- PSEL saves 743.6 of 15,516.0 HQC or 4.79 %.
- The median saving per circuit is 3.94 %.
- hwb8 alone gives 84.5 % of the saving and serves no sourced quantum algorithm. Without hwb8 the saving is 115.0 HQC.

Assessment [O]. The median is the better guide for a single small circuit.

Worked use case on Grover search with 7 and 9 qubits

Two complete Grover searches with one marked item, built from Grover 1996 and Barenco et al. 1995. Both pass an exact check under P0 and P2.

Instance	Qubits	HQC P0	HQC PSEL	Saving %	Shots P0	Shots PSEL	Extra shots
grover_d5	7	734.2	694.4	5.42	13699	14490	791
grover_d6	9	1629.4	1548.6	4.96	6153	6475	322

Shots within a 10,000 HQC budget, at most 10,000 shots and 5 HQC per job.

- PSEL buys 5.77 % more shots on grover_d5 and 5.23 % more on grover_d6.
- The T-count falls from 448 to 210 and from 1008 to 436. No platform job ran these instances.

Assessment [O]. The T-count drop comes from Toffolis that the Barenco construction repeats on the same qubits. It is not a general ZX gain.

A certified floor bounds what exact T optimisation can still gain

- Each T gate of a surface-code machine consumes one magic state. On H2 a T gate is a free Rz.
- On B1 the ZX stage removes 2,862 of 7,048 T gates of the written circuits, or 40.6 %.
- The floor is the channel bound of Gosset et al. (2014). On 20 synthesised Haar circuits of 2 and 3 qubits it is 0.997 of the T-count after ZX, pooled. The worst instance keeps at least 2478 of its 2496 T gates.
- Further exact T optimisation can gain at most 0.3 % pooled and 0.72 % on the worst instance.
- On the four certified B1 circuits the six published optimisers close 1 of the 59 gates between our count and the floor.
- The certificate covers 24 circuits with at most 5 qubits. It covers no circuit of family C and no Grover instance.

Assessment [O]. The certificate cannot tell whether the other 58 gates are headroom for a better rewrite or slack in the bound.

The vbe_adder_3 gain misses 0.05 after correction

Circuit	Pipeline	Two-qubit gates	Pooled F_H	sd	Per-run F_H range
vbe_adder_3	P0	56	0.9520	0.0034	0.9470 to 0.9560
vbe_adder_3	P2	41	0.9628	0.0030	0.9590 to 0.9680
grover_5	P0	240	0.8704	0.0053	0.8482 to 0.8652
grover_5	P2	218	0.8711	0.0053	0.8400 to 0.8728
vbe_adder_7 (22 qubits)	P0	142	0.886	0.0101	1000 shots, one run
vbe_adder_7 (22 qubits)	P2	107	0.897	0.0096	1000 shots, one run

H2-2E. First four rows 4000 shots per pipeline, Hellinger fidelity F_H of the summed counts.

- vbe_adder_3. F_H(P2) minus F_H(P0) is $+0.0108$, $z = +2.38$, two-sided $p = 0.017$.
- Pooled Holm p is 0.052 over three circuits. Repeats alone give $z = +2.44$, repeats-only Holm $p = 0.030$ over two. Inputs are all-zero.
- vbe_adder_7 of family C runs on H2-2E. Its difference of $+0.0110$ ($z = 0.79$) is not resolved at 1000 shots.
- 20 H2-2E jobs used 6598.0 HQC by the cost formula. Nothing has run on hardware.

Assessment [O]. The pooled test is primary. The repeats-only check reaches 0.030 but was chosen after the pooled result was known.

The HQC saved has a dollar value at Azure plan averages

- Azure Quantum lists the H2 Standard Plan at USD 125,000 per month with 10,000 HQC and the Premium Plan at USD 175,000 with 17,000 HQC.
- The plan-average price is USD 12.50 per HQC on Standard and USD 10.29 on Premium.

[O] Saving	HQC	USD at plan averages
Family C, 15 circuits without continuous angles	748.4	about 7,700 to 9,400
Family A, avoided loss against always running P2	1,076.4	about 11,100 to 13,500

- Both figures count 1000 shots of each circuit once.
- They use the plan average and not a marginal price. They ignore the bundled eHQCs and the Azure infrastructure costs.

Assessment [O]. The USD values rest on plan-average prices and not on a quoted marginal price.

Limitations and the plan for H2 hardware in the finals

Limitations

- No user interview or survey is part of this submission.
- Family C is count-only apart from one H2-2E run. PSEL output proven on 18 of 20, PSEL4 on 17.
- Nothing has run on H2 hardware. All platform results are emulator runs.
- This submission contains no learned rewriting.
- PSEL costs 8.37 % more than pytket level 3 on family C. PSEL4 adds level 3 as a candidate and saves 0.80 % against it on 17 proven circuits.

Finals plan

Test	Shots	Power	HQC
vbe_adder_3, recommended	4364	0.80	1147.3
vbe_adder_3, minimum	2900	0.65	765.7
grover_5, secondary	1900	0.65	2069.2

Shots per pipeline, P0 and P2.

- **[O]** Power 0.80 holds if the emulator gain holds on hardware.
- **[O]** A one-sided z of 1.645 or more confirms the gain.

Every result can be rerun from the code package

- `code.zip`. The source of the repository. `README.md` gives the run instructions.
- `requirements.txt`. Pins 20 packages, among them pytket 2.18.4, pytket-quantinuum 0.59.3 and pyzx 0.10.7.
- `run_tests.sh`. Runs 27 test files, eight as scripts and nineteen under pytest. All 27 return rc=0 from a fresh unzip of `code.zip`.
- `EVIDENCE.md`. Lists every emulator job with its job ID, shots, HQC estimate and stored payload.
- **Generated tables.** `report/make_tables.py` builds the results tables from `results/*.csv`. The circuit generators use seed 20261006.
- **Independent checks.** Verification reports by agents that did not produce the results are in `results/`.